

Relatório 11 - Prática: Lidando com Dados do Mundo Real (II)

Matheus Beuren Carvalho

1. Introdução

O objetivo desta aula, foi aprender e praticar as principais formas de lidar com os dados na engenharia de dados, que por sua vez, tem como objetivo limpar, transformar e preparar os dados para treinos de algoritmos de Machine Learning. A seguir, todo o conteúdo do curso em inglês disponibilizado pelo card.

2. Desenvolvimento

Em primeiro plano, conheci o primeiro modelo de Machine Learning do curso, ele é o mais simples que será mostrado. O KNN (K-Nearest-Neighbors), é um algoritmo que mede a distância entre de um ponto X e seus vizinhos comparando features e ao final calculando uma média de score, o mostrado no vídeo era relacionado a filmes, onde comprava uma lista binária com gêneros, e a popularidade dos filmes. Neste caso, o KNN funciona como regressão pois ao final é calculado a média de avaliação. Porém, para isso acontecer, foi necessário uma normalização no dataset para juntar as avaliações dos mesmos filmes e assim fazer uma média de avaliação para cada e também juntar o número de avaliações. A parte prática deste modelo foi praticamente a mesma, porém foi feito testes com números diferentes de vizinhos (K), como está na (Tabela 1).

Tabela 1 - Testes número de vizinhos filme Toy Story

K	Média de avaliação
2	3.48
5	3.71
10	3.34
15	3.46
20	3.49
50	3.38
Todos os Filmes	3.07

Analizando os resultados da Tabela 1, podemos perceber que $K=5$ alcançou uma média de avaliação mais alta que $K=50$ e $K=N$ números, nisso quanto maior K , menor o grau de similaridade, fazendo assim uma recomendação menos específica. Um K maior inclui vizinhos mais distantes no cálculo da média, o que diminui a influência dos vizinhos realmente parecidos.

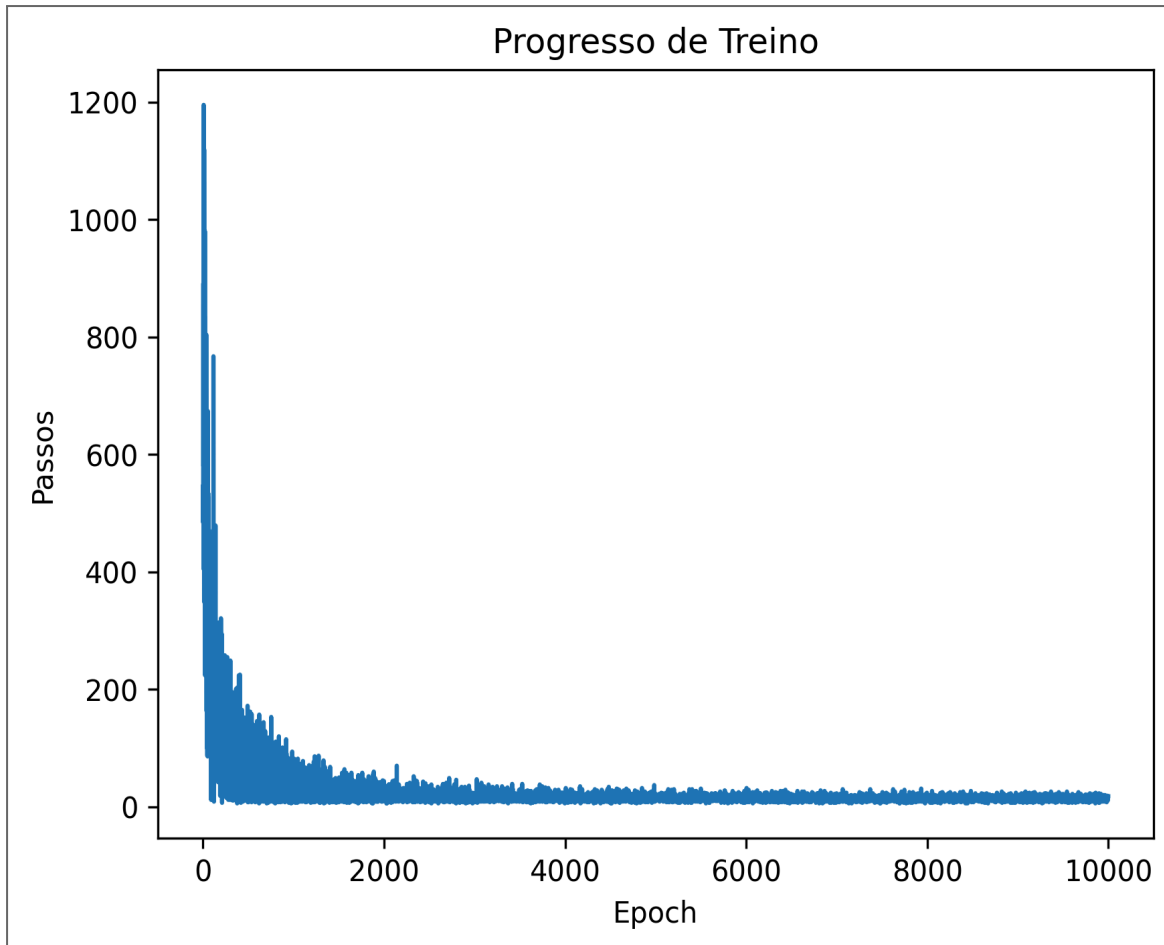
O PCA é uma forma de transformar um dataset de várias colunas em um conjunto menor onde pode ser visto a variância entre os dados, de preferência em um gráfico de dispersão. No exemplo do vídeo foi usado o dataset Íris, onde após a transformação do PCA foi observado que os pontos das diferentes espécies de flores, ficaram separados no gráfico de dispersão, mostrando que a variância que diferencia as espécies foi preservada pela transformação.

Data Warehousing é uma forma de juntar os dados brutos de diferentes fontes para que, possa ser possível cruzar esses dados para obter uma análise sobre determinado comportamento entre eles. Porém, para popular o Data Warehousing, podemos utilizar o ETL(Extract Transform Load) para popular o banco que armazena os dados dessa fonte(Warehousing). A diferença entre ETL e ELT, se baseia basicamente quando a transformação dos dados acontece, ELT seria usado com "hadoop" que é uma ferramenta de processamento distribuído suportando altos níveis de dimensões e dados complexos, dando menos custo a transformação dos dados.

Aprendizado por reforço, significa que o algoritmo aprenderá conforme explora e assim vai recebendo recompensas baseadas nas decisões tomadas. Para isso, o “Estado” é o ambiente em que o algoritmo está, “Recompensas” seriam os valores negativos ou positivos acrescentados a pontuação do algoritmo, fazendo-o assim aprender o melhor caminho com base em recompensas. O Q-learning, é o valor associado a cada estado/ação, como por exemplo o da recompensa a cada ação feita no ambiente. Com o tempo, o algoritmo passa a ter referências do que fazer e o que não fazer determinado ao seu estado atual e as ações possíveis.

Seguindo o exemplo do instrutor, treinamos um algoritmo para jogar um minigame de táxi, onde o táxi tem que pegar e entregar a pessoa. Foi feito um treino com 10000 epochs, nisso eu fiz um gráfico para melhorar a visualização de quanto passos o algoritmo deu para aprender os melhores caminhos, sendo que um número menor de passos para fazer a corrida melhor, do que muitos para somente uma corrida. No gráfico a seguir pode ser visto como os passos diminuíram ao longo do treino para o algoritmo aprender o melhor caminho.

Figura 1 - Passos por epoch



Analisando o gráfico, podemos ver que o desempenho do modelo antes de 2000 epochs foi relativamente alto, pois o algoritmo estava aprendendo o jogo, após isso percebemos que o modelo começou a tomar as melhores decisões sempre.

A matriz de confusão vai além da acurácia de um modelo, ela considera tipos de resultados:

- Verdadeiro Positivo: o modelo previu sim e era sim;
- Falso Positivo: o modelo previu sim e era não;
- Falso Negativo: o modelo previu não e era sim;
- Verdadeiro Negativo: o modelo previu não e era não.

Dando um exemplo de fraude bancária, onde 99,5% das transações são normais, e 0,5% são fraudes, a acurácia do modelo é 99,5% o modelo somente prevê que ela não é uma fraude em todos os casos, tendo assim um modelo inútil. Nesse contexto surge a matriz de confusão, ela alinha os tipos de resultados em uma matriz, que mostra em sua diagonal os testes que deram certo e os demais erros que o modelo cometeu, pois a diagonal é o cruzamento das mesmas classes, sendo uma a de predição e a outra da resposta correta.

Em métricas de avaliação, temos seis principais métricas:

- Recall: (Verdadeiro Positivo / (Verdadeiro Positivo + Falso Negativo)) é usado quando se valoriza os Falsos Negativos;
- Precision: (Verdadeiro Positivo / (Verdadeiro Positivo + Falso Positivo)) utilizado quando o Falso Positivo importa;
- F1 Score: $(2 * ((Precision * Recall) / (Precision + Recall)))$ usado junto com precision e recall;
- RMSE: pega a diferença entre a previsão e a real, eleva ao quadrado, tira a média, e depois a raiz quadrada, se importando apenas com o acerto ou erro;
- ROC: um gráfico que compara Verdadeiro Positivo no eixo Y com Falso Positivo no eixo X, quanto mais o alinha se curva para o canto superior esquerdo melhor o modelo;
- AUC: é a probabilidade do modelo dar uma pontuação maior para um positivo aleatório do que para um negativo. Métricas:
 - AUC = 1.0 classificador perfeito;
 - AUC $\leq$ 0.5 o modelo é inútil, chutou algum número.

O Bias Variance, se distingue em duas métricas, sendo elas o Bias que mede o quanto as previsões do modelo estão longe da resposta correta, e a Variance que significa o quão espalhados estão os dados. A seguir na (Tabela 2) é mostrado as combinações das métricas para avaliar as previsões do modelo.

Tabela 1 - Métricas de desempenho

	Variance Alto	Variance Baixo
Bias Baixo	As previsões estão perto do correto mas estão muito espalhadas	Melhor resultado, as previsões estão perto do correto e sem muita variância
Bias Alto	Pior cenário, as previsões estão longe do acerto e muito espalhadas	Previsões longe do acerto mas pouco distribuídas

A fórmula de erro total: $Erro = Bias^2 + Variância$.

O K-fold resolve o problema da generalização dos dados novos, ou seja, quando o modelo não consegue ter certeza da sua previsão, ele dá um chute aleatório, isso com a separação tradicional em dados de treino e teste somente uma vez. O K-fold vem para solucionar este problema, separando cada vez o dataset inteiro em uma quantidade pré definida, e usa uma parte para teste e as outras para treino, nisso retorna um array de N acurácias individuais.

A limpeza de dados é uma das partes se não a parte mais importante em ciência de dados, pois ela é quem define se um modelo pode ser ruim ou muito bom. Não ter um processo de limpeza pode fazer com que um modelo que tenha dados limpos e tratados seja muito mais eficaz que um modelo sem limpeza. Nesse contexto, temos a normalização, é uma forma de representar números de uma maneira mais fácil, por exemplo, uma medida que vai de 1 até 1 bilhão, a normalização tratará para representar esse intervalo em 0 a 1 por exemplo.

Os outliers são um problema para dados em ciência de dados. Significam dados muito distantes da média, ou seja, um dado muito maior que os demais, nisso esses outliers precisam ser tratados pois com eles a média e todas as operações são interferidas.

Feature Engineering é o processo de seleção de features (colunas), transformá-las, normalizar e fazer o tratamento. Usar todas as features para treinar um modelo é totalmente inviável, pois a cada feature que o modelo recebe é uma dimensão para o modelo achar padrões, então é necessário realizar a análise de quais features realmente importam para a predição. Também é comentado no vídeo que, o processo de seleção é retomado várias vezes após o treino seja por uma feature que não está ajudando, ou por um resultado não agradável.

O tratamento de dados faltantes é um processo muito importante, pois nem todo dataset terá dados perfeitos, sempre terá um buraco para tapar. Nisso, se não tiver a opção de coletar mais dados deve-se optar por esse tratamento, alguns exemplos são:

- Incluir a média: nesse caso, significa fazer uma média da coluna e colocar no dado faltante (sensível a outliers);
- Descartar a linha: em últimos casos a remoção da linha pode ser viável, mas sempre recomenda-se não fazer isso principalmente se não tem a possibilidade de uma coleta de mais dados;
- Implementar a mediana: obter a mediana da coluna e colocá-la no dado faltante;
- Machine Learning: é usado um modelo para prever um dado para substituir o dado faltante. Usado quando tem muitos dados faltantes.

Em questão da separação dos dados de treino para um modelo, um assunto muito importante é o balanceamento dos dados. Nada adianta, todas as etapas serem perfeitas mas na hora do modelo ser treinado, tem somente poucos casos de uma predição específica, por exemplo, a detecção de fraudes onde é analisado milhares de transações sendo a maioria não fraudes, tendo assim poucos casos onde realmente é fraude, assim o modelo pode decorar e quase nunca prever uma fraude, também chamado de Falso Negativo. Por isso, abordagens como oversampling, undersampling, SMOTE e threshold cuidam disso para que seja implementado ou criado dados para a classe minoritária.

Em alguns modelos de Deep Learning, é necessário a entrada por categorias, nisso é usado o One-Hot Encoding, transformando os dados em dados categóricos, por exemplo, a coluna tem cinco entradas, o modelo coloca como um na categoria correta e o resto fica como zero. Além disso, existem outros métodos citados na aula para normalizar e ajudar o modelo a interpretar melhor os dados.

3. Conclusão

Os conteúdos que foram abordados nesse card resolvem os principais problemas encontrados na parte de tratamento dos dados e a preparação dos dados para os treinos de Machine Learning e Deep Learning. Os casos de prática são muito úteis para o entendimento dos conteúdos e sua importância para o que vem a seguir no Bootcamp.